

Plaid Bank Payment Implementation Guide

WizloPay Custom Checkout · wizlopay-custom-checkout

June 2026

1. Overview

Plaid is integrated via gr4vy as the bank-link payment method. gr4vy acts as the orchestration layer: it generates the Plaid Link token via its payment service definition API and then creates the transaction using the Plaid public token returned by the Plaid Link UI. The frontend never handles credentials or payment service IDs directly.

Integration Flow

```
User clicks "Pay with bank"
  |
  v
Frontend -> POST /api/checkout/plaid-session
  |
  v
Backend -> gr4vy paymentServiceDefinitions.session('plaid-bank')
  | Returns: { linkToken }
  v
Frontend opens Plaid Link UI (modal)
  |
  v
User selects bank -> authenticates
  | Plaid returns: publicToken + accountId
  v
Frontend -> POST /api/checkout/transaction
  | { method: "plaid", token: publicToken, accountId }
  v
Backend -> gr4vy transactions.create()
  | (auto-injects paymentServiceId from env)
  v
gr4vy returns transactionId -> redirect to callback
```

2. Prerequisites

- gr4vy account with Plaid payment service enabled in the Dashboard
- WIZLOPAY_PLAID_SERVICE_ID: the UUID of the Plaid payment service from gr4vy Dashboard
- WIZLOPAY_INSTANCE_ID and WIZLOPAY_PRIVATE_KEY already configured
- Plaid Link JS SDK loaded from CDN (handled automatically by PlaidButton component)
- No Plaid API keys required on the frontend

3. Environment Variables

Variable	Where	Req.	Description
WIZLOPAY_PLAID_SERVICE_ID	apps/backend/.env	Yes	UUID of the Plaid payment service from gr4vy Dashboard. Found under Connections → Payment Services.
WIZLOPAY_INSTANCE_ID	apps/backend/.env	Yes	gr4vy instance ID (shared with frontend)
WIZLOPAY_PRIVATE_KEY	apps/backend/.env	Yes	gr4vy private key for JWT signing
NEXT_PUBLIC_WIZLOPAY_INSTANCE_ID	apps/frontend/.env.local	Yes	Same instance ID, exposed to Next.js

4. Backend Implementation

4a. Get Plaid Link Token — getPlaidLinkToken()

File: *apps/backend/src/payments/payments.service.ts*

Key implementation notes:

- Uses `paymentServiceDefinitions.session()` (NOT `paymentServices.session()`)
- The fixed service definition ID is always 'plaid-bank' — this is gr4vy's built-in identifier
- The action 'create-link-token' tells gr4vy to call Plaid's `/link/token/create` on your behalf
- The `link_token` is extracted from `session.responseBody['link_token']`

```

async getPlaidLinkToken(): Promise<{ linkToken: string }> {
  const session = await this.jwtService.client
    .paymentServiceDefinitions
    .session(
      { action: 'create-link-token' },
      'plaid-bank' // fixed gr4vy identifier -- not a UUID
    );

  const linkToken = session.responseBody?.['link_token'] as string | undefined;
  if (!linkToken) throw new Error('No link_token returned from Plaid session');
  return { linkToken };
}

```

4b. Create Plaid Transaction — createTransaction()

File: *apps/backend/src/payments/payments.service.ts*

Key implementation notes:

- method must be exactly 'plaid'
- token is the Plaid `publicToken` from the Link UI's `onSuccess` callback
- `accountId` is `metadata.account_id` from the same callback
- `paymentServiceId` is required by gr4vy for Plaid — auto-injected from `WIZLOPAY_PLAID_SERVICE_ID`
- intent: 'capture' is required to immediately capture funds (not just authorise)

```
// In createTransaction(), when method === 'plaid':
if (!resolvedPaymentServiceId && params.paymentMethod['method'] === 'plaid') {
  resolvedPaymentServiceId = this.config.get<string>('WIZLOPAY_PLAID_SERVICE_ID');
  if (!resolvedPaymentServiceId) {
    throw new BadRequestException(
      'WIZLOPAY_PLAID_SERVICE_ID is not set. ' +
      'Add the Plaid payment service UUID from your gr4vy Dashboard.'
    );
  }
}

await this.jwtService.client.transactions.create({
  amount: params.amount,
  currency: params.currency,
  country: params.country,
  paymentMethod: {
    method: 'plaid',
    token: publicToken,          // from Plaid Link onSuccess
    accountId: accountId,        // from Plaid Link metadata
  },
  buyer: { id: buyerId },
  intent: 'capture',
  paymentServiceId: resolvedPaymentServiceId, // auto-injected
});
```

4c. Controller Endpoints

File: apps/backend/src/payments/payments.controller.ts

- POST /checkout/plaid-session — calls `getPlaidLinkToken()`, returns `{ linkToken }`
- POST /checkout/transaction — calls `createTransaction()` with the Plaid payload

5. Frontend Implementation

5a. PlaidButton Component

File: apps/frontend/components/checkout/PlaidButton.tsx

Step-by-step flow:

1. On mount: Load Plaid Link JS from <https://cdn.plaid.com/link/v2/stable/link-initialize.js> (idempotent — checks for existing script tag)
2. On click: POST /api/checkout/plaid-session with gr4vy JWT token in Authorization header → receives `linkToken`
3. Call `window.Plaid.create({ token: linkToken, onSuccess, onExit })` → call `handler.open()`
4. onSuccess(`publicToken`, `metadata`): POST /api/checkout/transaction with `{ method: 'plaid', token: publicToken, accountId: metadata.account_id, intent: 'capture' }`
5. On success: call `onSuccess(transactionId)` which redirects to /checkout/callback

Code — key handler:

```
const handler = window.Plaid.create({
  token: body.linkToken,
  onSuccess: async (publicToken, metadata) => {
    const txRes = await fetch('/api/checkout/transaction', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`, // gr4vy JWT
      },
      body: JSON.stringify({
        amount, currency, country,
        paymentMethod: {
          method: 'plaid',
          token: publicToken,
          accountId: metadata.account_id,
        },
        intent: 'capture',
        ...(buyerId && { buyerId }),
      }),
    });
    const { transactionId } = await txRes.json();
    onSuccess(transactionId);
  },
  onExit: (err) => { if (err) console.error(err); },
});
handler.open();
```

5b. CheckoutShell Integration

PlaidButton is rendered in the "Bank" tab of CheckoutShell:

```
<PlaidButton
  token={checkout.token}
  amount={amount}
  currency={currency}
  country={country}
  buyerId={buyerId}
  onSuccess={onSuccess}
  onError={onError}
/>
```

6. API Route (Next.js Proxy)

The frontend calls `/api/checkout/*` which Next.js proxies to the NestJS backend. The relevant routes are:

```
/api/checkout/plaid-session → POST http://backend:3001/checkout/plaid-session
/api/checkout/transaction  → POST http://backend:3001/checkout/transaction
```

7. Error Handling

Scenario	Error / Symptom	Fix
WIZLOPAY_PLAID_SERVICE_ID not set	"WIZLOPAY_PLAID_SERVICE_ID is not set. Add the Plaid payment service UUID from your gr4vy Dashboard."	Add UUID from gr4vy Dashboard → Connections → Payment Services to <code>apps/backend/.env</code>

Scenario	Error / Symptom	Fix
Plaid service not enabled in gr4vy	gr4vy API 400 validation error	Enable the Plaid payment service in the gr4vy Dashboard
gr4vy "payment_service_id is required"	BadRequestException from gr4vy API	WIZLOPAY_PLAID_SERVICE_ID is set but incorrect UUID; verify in Dashboard
Plaid Link SDK not loaded	"Plaid SDK not loaded yet"	Check browser network tab for CDN load failure; verify no CSP blocking cdn.plaid.com
No link_token in session response	"No link_token returned from Plaid session"	Plaid service definition session call failed; check gr4vy logs
User exits Plaid Link	onExit fires (no error shown)	Normal flow; user can retry

8. Testing

Sandbox Testing

1. Ensure NEXT_PUBLIC_WIZLOPAY_ENVIRONMENT=sandbox in your frontend .env.local
2. Click the "Bank" tab in the checkout UI
3. Click the "Pay with bank" button
4. In the Plaid Link UI, use sandbox test credentials: Institution: any sandbox bank (e.g. "First Platypus Bank") Username: user_good Password: pass_good
5. Select a checking account and confirm
6. Verify redirect to /checkout/callback with transaction_id and transaction_status=capture_succeeded

Verification Checklist

- [] WIZLOPAY_PLAID_SERVICE_ID is set in apps/backend/.env
- [] Plaid payment service is enabled in gr4vy Dashboard
- [] POST /api/checkout/plaid-session returns { linkToken }
- [] Plaid Link UI opens after clicking the button
- [] onSuccess fires with publicToken and account_id
- [] POST /api/checkout/transaction succeeds with { transactionId }
- [] Redirect to /checkout/callback with correct params
- [] Orders table has new row with wizlopayTransactionId set

9. Common Gotchas

- Use paymentServiceDefinitions.session() not paymentServices.session() — these are different gr4vy API methods
- The first argument to .session() is the options object ({ action: 'create-link-token' }), the second is the fixed string 'plaid-bank'
- Never expose WIZLOPAY_PLAID_SERVICE_ID to the frontend — it is injected server-side only
- The Plaid method requires paymentServiceId on every transaction — gr4vy will reject without it

- intent: 'capture' must be set explicitly for bank transfers — do not omit it
- The Plaid Link JS CDN URL must not be blocked by Content Security Policy; add `cdn.plaid.com` to your CSP `script-src`

End of Plaid Bank Payment Implementation Guide — WizloPay Custom Checkout